

SIMT-Aware Lockstep Verification and Functional-Coverage Closure Methodology for an Open-Source RISC-V GPGPU: A UVM 1.2 Environment

Samuel Moussa*, Steven Ibrahim*, Ahmad Sudky*,
Ahmad Fawzy*, Abanoub Nabil*, Alhassan Sayed*,
Hossam Hassan†, and Hyung-Min Yoon‡

*Department of Electronics and Communications Engineering,
Faculty of Engineering, Minia University, Egypt

†Independent Researcher, South Korea ‡Siliconarts, Inc., South Korea
Corresponding author: samuelmoussa64@gmail.com

Abstract—Open-source RISC-V GPGPUs such as Vortex ship with directed-kernel regressions but no reference-model checking, functional-coverage model, or sign-off discipline. This paper presents a UVM 1.2 environment and methodology that closes that gap. The environment wraps a bus-master SIMT DUT with role-inverted agents, integrates Vortex’s functional simulator (SimX) as a per-configuration golden model over DPI-C, and renders verdicts via two injection-qualified checkers: a bidirectional end-state scoreboard and a per-instruction, per-lane lockstep comparator under five SIMT alignment rules. A two-pass load-value feed makes racy fenceless multi-core programs instruction-granularity verifiable (residual zero over 5,432 retirements), with the interrupt-timing boundary stated. A three-layer coverage model adds, to our knowledge, the first published SIMT functional-coverage layer for RTL GPU verification (divergence depth, bank conflicts, coalescing classes), closing its own layers at 98.1% coverage-bin / 94.7% total (ISA layer separately: 83.1% bins / 89.3% weighted) under machine-generated, RTL-cited exclusions and a blocking waiver-integrity gate; an unstimulated D-extension elaboration affects no functional bin, only lowering totals. The checking depth surfaced real defects on both sides of the comparison: a JALR LSB ISA deviation, a non-scaling watchdog constant (since fixed upstream), a reset-relay X window found by restoring a silenced assertion, a missing AXI error path proven by fault injection, and a reference-model fetch bug found by the lockstep itself. FuzzGPU (USENIX Security 2026), a concurrent RTL GPU fuzzer on the same DUT, is complementary: fuzzing finds bugs, this methodology quantifies sign-off; both independently found the JALR deviation. All findings ship in an evidence-cited register; every number carries provenance and the method’s boundaries are stated rather than waived.

Index Terms—UVM, RISC-V, GPGPU, SIMT, functional verification, lockstep co-simulation, functional coverage, coverage closure, fault injection, open-source hardware

I. INTRODUCTION

Open-source silicon has an industrial verification problem. The RISC-V ecosystem has produced mature, publicly documented verification assets for *scalar* cores—step-and-compare lockstep environments, constrained-random instruction gen-

erators, ISA coverage libraries [4]–[7], [9]—but the most complex open processor class, the GPGPU, has none of this. Vortex, the leading open RISC-V GPGPU [1], [2], ships with a directed-kernel regression and CI, no reference-model checking at the RTL level, and no functional coverage model. Every consumer of Vortex—research forks, GPGPU-PG studies, accelerator prototypes—inherits that gap. Concurrently, FuzzGPU [3] demonstrated that attackers and fuzzers can find real bugs in exactly this design space (20 previously unknown bugs, 10 CVEs, across Vortex and Ventus), which converts the verification gap from an academic inconvenience into an exposure.

This paper presents a UVM 1.2 [10] environment and methodology that closes that gap for Vortex, and reports what the resulting checking depth revealed about both the RTL and its reference model. Our contributions:

- 1) **A complete, configurable UVM environment for a SIMT GPGPU** (Section III): role-inverted agents (DUT = AXI master, agents = reactive slaves), a DPI-C–integrated functional golden model rebuilt per configuration, a passive RVVI-style observability layer, and a bidirectional end-state scoreboard—any *clusters* × *cores* × *warps* × *threads* topology from runtime options (plusargs) with elaboration-time parameter self-checks.
- 2) **Per-instruction lockstep for SIMT** (Section V): five alignment rules—uuid-keyed program-order sorting, mask-union record aggregation, per-lane compare scoping, and volatile-CSR exclusion—that make CPU-style lockstep sound for a SIMT machine, validated lane-exact at five configurations (six programs) spanning a $2 \times 2 \times 3 \times 2$ axis space.
- 3) **A sound two-pass load-value feed** (Section VI) that makes racy, fenceless multi-core programs—architecturally undefined and previously unverifiable—instruction-granularity verifiable, with the *residual as*

the verdict and non-vacuity proven by injection; plus a characterized soundness boundary at asynchronous interrupt timing (Section XI).

- 4) **A three-layer coverage model with honest closure** (Section VIII): a third-party ISA layer, an original SIMT-microarchitecture layer—the first published functional-coverage model of warp divergence depth, scratchpad bank conflicts, and memory-coalescing classes for RTL GPU verification—and a protocol/assertion layer; closed at 98.1%/94.7% (covergroup bins/total) on the environment’s own microarchitecture and protocol layers—the third-party ISA layer closes separately at 83.1% bins / 89.3% weighted after register-index exclusions—using only machine-generated, per-configuration, RTL-cited structural exclusions under a blocking waiver-integrity gate, with unreachable-versus-unhit distinguished by evidence and structural ceilings root-caused rather than waived.
- 5) **An evidence-classified findings register** (Sections IX–X): RTL bugs, hazards, and observability limits, and reference-model defects—each with `file:line` evidence and a disposition—including a reference-model fetch bug found by the lockstep, demonstrating that a lockstep flow verifies the golden model as much as the DUT.

Section II positions the work against concurrent and prior art; Section XI delimits the method’s own soundness boundary; Section XII separates what front-end functional verification can claim from everything silicon sign-off would still require.

II. BACKGROUND AND RELATED WORK

A. Vortex

Vortex [1], [2] is an open-source RISC-V GPGPU organized as clusters → sockets → cores → warps → threads, with per-core instruction/data caches, optional shared L2/L3, an immediate-post-dominator (IPDOM) reconvergence stack for divergence, and hardware barriers. Execution units comprise ALU, FPU (FPnew), LSU, SFU (CSR/warp control), and a tensor unit (TCU). The verified build is RV32IMAF at RTL pin 7a52ee5, primary configuration 1 cluster / 1 core / 4 warps / 4 threads (“1CL/1C/4W/4T”) with an AXI memory interface, simulated on QuestaSim 2021.2 [23]. Notably, Vortex has *no trap architecture*: there are no illegal-instruction, misaligned-address, or CSR exceptions—a property with verification consequences developed in Section IX.

B. Reference models

Vortex ships SimX, a functional (non-timing) simulator maintained with the RTL. SimX’s emulator steps per instruction, returns a retirement trace with full SIMT architectural state, and is decoupled from its timing model—structurally an RVVI-shaped golden model [6]. Spike [8] cannot execute the six SIMT instructions, so SimX is the primary golden model and Spike serves as a *secondary, independent* base-ISA cross-check. That audit has been carried out: on the

same linked ELF, Spike, SimX and the DUT retire exactly 11,076 architectural writebacks and agree on every program counter, destination register and value—zero mismatches—with non-vacuity proven by injecting a fault at one record and confirming it is named. Its scope is bounded by construction and we do not overstate it: Spike is a scalar ISS, so the audit covers warp 0 / lane 0 base-ISA execution only and stops at the first Vortex custom operation. **The SIMT axis therefore has no independent reference**, and will not obtain one from this direction. The consequence—that the primary golden model is maintained by the same project and must itself be treated as a verification target—is examined in Section X.

C. Lockstep and reference-model flows

Step-and-compare verification against a reference model is the standard of care for RISC-V CPU cores: core-v-verif [5] established the open-source pattern with ImperasDV [9] over the RISC-V Verification Interface [6]; riscv-dv [7] supplies constrained-random programs and a trace-compare co-simulation flow; DiffTest [4] industrialized per-instruction architectural-state comparison over DPI-C for the XiangShan processor line. Industrial-scale UVM methodology on open silicon is demonstrated by OpenTitan [22]. All of these target scalar (or SIMD-vector) CPUs. Our environment descends from this methodology family and extends it in three directions: to a bus-master DUT (role-inverted agents), to SIMT semantics (per-lane masked compare, divergence, multi-record retirements), and to weakly-ordered multi-core programs (the two-pass load feed). Per-warp differential testing on this very DUT has since also been demonstrated by FuzzGPU (below); the lockstep layer presented here differs in its formalized alignment rules, its UVM packaging, and its role as one of several qualified checkers inside a sign-off flow rather than a fuzzer’s oracle.

D. GPU verification and fuzzing

Software-level GPU kernel verification is a mature line: GPUVerify [14] proves race- and divergence-freedom of OpenCL/CUDA kernels; GKLEE [15] performs concolic testing and explicitly targets divergent warps, non-coalesced accesses, and shared memory bank conflicts as bug classes. These operate on kernel source, not RTL; producer-consumer synchronization has been verified in GPU programs [21], learned program generation has precedent in compiler fuzzing [16], and formal memory-consistency verification reaches the RTL level in RTLCheck [19] and the PTX memory-model formalization [20]. At the hardware level, FuzzGPU [3] (USENIX Security 2026) is the first RTL GPU fuzzer: it generates valid SIMT programs (divergence-aware generation stack, deadlock-free barrier planner, interleaved warp generation with taint propagation), runs them on Verilator [24] models of Vortex and Venus, and performs trace-driven differential testing against ISA-level instruction-set simulators with per-warp state, reporting 20 previously unknown bugs and 10 CVEs. Its evaluation is structural-coverage-based (line/toggle/expression), its campaigns are time-boxed, and it carries no functional-coverage

TABLE I

POSITIONING AGAINST REFERENCE-MODEL VERIFICATION AND GPU TESTING FLOWS. “SIMT SEMANTICS” COVERS PER-LANE MASKED COMPARE AND SPLIT-RETIRE AGGREGATION; “CHECKER QUALIFICATION” MEANS FAULT-INJECTION GUARDS; “WAIVER DISCIPLINE” MEANS MACHINE-GENERATED, GATE-CHECKED EXCLUSIONS. ISA COVERAGE FOR OUR L1 LAYER IS PROVIDED BY THE THIRD-PARTY RISCVISACOV LIBRARY; THE SIMT MICROARCHITECTURE MODEL IS THIS WORK’S CONTRIBUTION.

Property	[5], [9]	[4]	[3]	This work
Verification target	scalar CPU	scalar CPU	RTL GPGPU	RTL GPGPU
Per-instruction compare	✓	✓	✓	✓
SIMT semantics	—	—	partial	✓
Racy-program two-pass feed	—	—	—	✓
ISA functional coverage	✓	—	—	✓
SIMT coverage model	—	—	—	✓
Checker qualification	—	—	—	✓
Waiver discipline	—	—	—	✓
Verdict taxonomy	—	—	—	✓
Framing	sign-off	agile DV	bug discovery	sign-off

model, no checker qualification, and no sign-off discipline—by design, since it is a bug-discovery engine. Our work is the complementary half of the problem: a UVM verification *methodology* whose claims are coverage-quantified, whose checkers are proven non-vacuous, and whose exclusions are machine-generated and gate-checked. The two approaches also cross-validate: the JALR LSB deviation we report as R1 (Section IX) was independently found by FuzzGPU (S2, upstream PR #339, CWE-682)—in the instruction-set-simulator component of the same project; their RTL findings (X1–X3, PRs #356/#358/#359) are disjoint from ours, consistent with the different pins and oracles involved. We have since exercised their reported Vortex findings at our pin: X1 reproduces (the PoC fires the RTL’s own invalid-writeback-register assertion), and X2—whose upstream fix landed entirely in the reference model’s decoder—reproduces as a genuine DUT-vs-SimX divergence in which the hardware is right and the golden model is wrong; both corroborate the two flows’ complementarity and, for X2, this paper’s thesis that the golden model is itself a verification target. Concurrent with both, the Ventus project developed GVM, a UVM-like verification framework for its own OpenGPGPU design; Vortex itself, to our knowledge, has had no public UVM environment prior to this work.

E. Checker qualification

Mutation-based qualification of checkers—proving a checker can fail by corrupting the design or its inputs—is established practice in CPU and SoC verification, both academically [17] and commercially; NVBitFI [18] brought systematic fault injection to GPUs for reliability studies. This work applies the discipline end-to-end to a SIMT lockstep flow: every verdict-rendering checker (Section IV) is guarded by permanent fault-injection regression tests, and the discipline is what allows the coverage and pass-rate numbers of Section VIII to be taken at face value. Table I summarizes the positioning of this work against the reference-model verification and GPU testing flows.

III. VERIFICATION ENVIRONMENT

Fig. 1 shows the environment: active agents drive the launch protocol and serve memory on the left, the functional reference runs in lockstep on the right, and a passive probe layer feeds the two checkers and coverage at the bottom.

A. Role-inverted agent architecture

A GPGPU DUT executes programs; it is not driven by them. The environment therefore inverts the usual UVM roles. Five agents surround the DUT: *host* and *device-configuration-register (DCR)* agents (active) drive the launch protocol—kernel base address, argument pointers, thread-count device-configuration registers—through the platform’s control interface; the *AXI* agent is an active *responder*, a reactive slave backed by a memory model that services the DUT’s fetch and data traffic; a *memory* agent covers the non-AXI configuration of the same model; and a passive *status* agent observes busy/completion state, the retired-instruction count, and the program counter. A virtual sequencer coordinates launch sequences. Stimulus is therefore a *program* (a compiled ELF), and the UVM test class controls only *how* it is launched, configured, and perturbed—tests and programs are orthogonal and composed at the build level.

B. Golden-model integration

SimX is linked into the simulation via DPI-C (`simx_init/load/dcr_write/run` plus a per-retirement record export). Both the RTL and SimX are rebuilt per configuration so that DUT and reference always agree on topology. Golden-model crashes are trapped by a signal handler and mapped to a sentinel exit code, so a reference-model failure is *classified* (Section IV) rather than crashing the run or silently passing it.

C. Passive observability layer

All white-box visibility is bound, passive, and never a checker:

- a *commit probe* bound to the retire arbiter of every core, capturing each retirement beat $\{uuid, wid, PC, rd, wb, tmask, per-lane\ data\}$ —where *uuid* is the RTL’s unique per-instruction identifier, *wid* the warp id, and *tmask* the active-thread mask—across all clusters, sockets, cores, and issue lanes;
- an *LSU writeback probe* bound to the load-store slice, capturing true per-lane load values after sign/zero extension (required because load data is not observable at the commit arbiter, Section IX);
- an RVVI-style interface and UVM monitor [6] that publishes the merged retirement stream through an analysis port;
- microarchitectural probes feeding the SIMT coverage layer: scheduler, divergence/reconvergence, local-memory bank, memory coalescer, cache-event, register-hazard, and commit-beat probes (Section VIII).

Lockstep capture is plusarg-gated; with the gate off, runs are proven byte-identical to the plain environment.

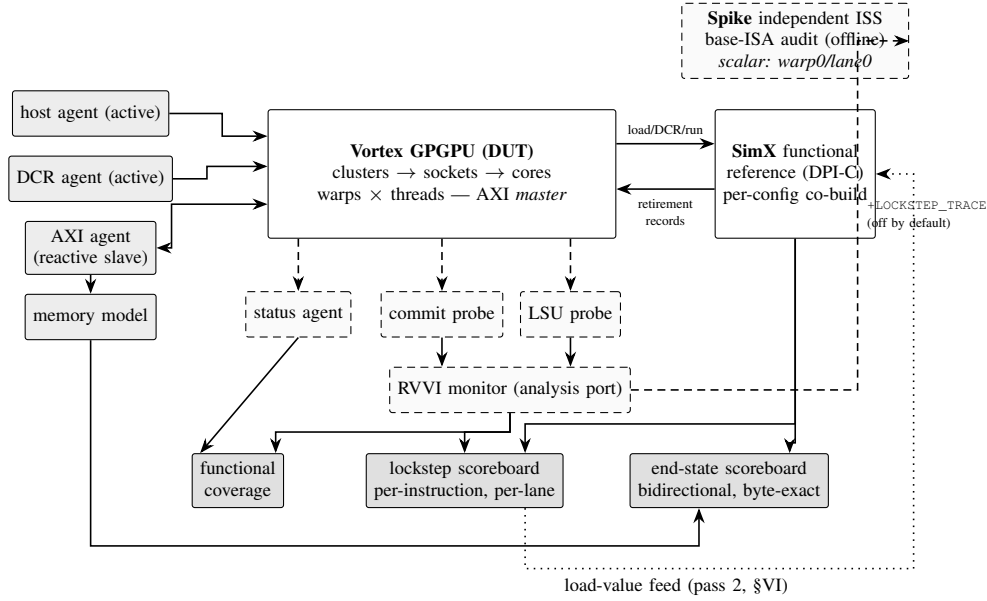


Fig. 1. Environment architecture. Solid gray boxes are active UVM agents; dashed boxes are passive probes and offline tools (never checkers in the run); dark boxes are the two verdict-rendering scoreboards and coverage. The DUT is the bus master; stimulus is a compiled program in the memory model. **Two references, with different roles:** SimX is the *primary* golden model, stepped live over DPI-C, and models SIMT; Spike is a *secondary, independent* scalar ISS used offline on an exported retirement trace to cross-check the base-ISA subset only (warp 0 / lane 0, stopping at the first custom op). SimX is co-designed with the RTL and is therefore not independent; Spike is independent but cannot execute a SIMT kernel. Neither substitutes for the other, and the SIMT axis has no independent reference (§XI).

D. Checkers

Two independent checkers render verdicts:

- 1) **Bidirectional end-state scoreboard.** After completion, every word the DUT wrote is compared byte-exact against SimX (forward pass, up to 196,868 words in one cache-tier test), *and* every word SimX wrote that the DUT never wrote is checked (reverse pass, catching dropped stores). Sub-word stores are handled with per-byte validity masking; IEEE-rounding-legitimate FP differences are tolerance-compared (and bounded, Section IX); read-only/GOT sections are excluded by region.
- 2) **Lockstep scoreboard** (Section V): every retired instruction compared per active SIMT lane.

A third, orthogonal gate makes the run verdict *assertion-aware*: any RTL runtime assertion firing fails the run with a distinct exit code, and failing runs are excluded from coverage merging. A dedicated negative test (a deliberate misaligned load whose value is discarded) keeps this gate honest: it must fail through the assertion path alone while the scoreboards pass.

E. Configurability with self-checks

One compiled environment runs any topology from plusargs (+CLUSTERS/+CORES/+WARPS/+THREADS); interface widths (e.g., the 50-bit memory tag) are derived from the RTL package, and elaboration-time assertions compare every UVM parameter against its DUT counterpart, failing loud on mismatch. Validated topologies include 1CL/1C/4W/4T, 1CL/2C, 2CL/2C/4W/4T, 4C/2W, and 8C/8W/2T.

F. Protocol assertions

An AXI4 SVA layer [11] (~15–18 protocol properties plus 11 handshake-stability assertions) checks burst legality, outstanding-count consistency, and signal stability under backpressure inline on the DUT’s AXI interface; slave-side *throttle* (ready wait-states) and *flood* (back-to-back R beats) stress modes—plusarg-gated and proven byte-identical when off—exercise the stability properties. An error-injection mode (+AXI_INJECT_ERR) drives SLVERR/DECERR responses on the read channel; its findings are reported as R9/OBS-057 in Section IX.

IV. VERDICTS AND NON-VACUITY

Every run renders one of four verdicts: PASS, FAIL, UNVERIFIABLE, or (for lockstep runs) END-STATE-VERIFIED with an instruction-granularity residual. UNVERIFIABLE is first-class: a run where the golden model cannot render a verdict (e.g., a reference-model abort) is classified with root-cause evidence—never force-compared, never silently dropped, and never counted as a pass.

Verdicts are only as good as their ability to go red. The environment therefore carries *permanent fault-injection guards* as regression tests, one per checker:

- **wrong-value injection** (+INJECT_FAULT): one bit of one DUT store is flipped; the end-state scoreboard must catch it;
- **dropped-store injection** (+DROP_STORE): one DUT store is suppressed; the reverse scoreboard pass must catch it (both guards historically detect at the exact address 0x800075d8);

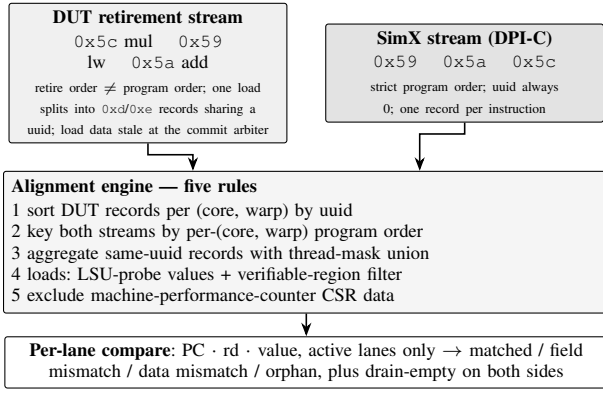


Fig. 2. The lockstep alignment pipeline. The DUT stream (left) carries the SIMT quirks that a scalar-CPU RVVI flow never meets; the reference stream (right) is strictly ordered. Each rule removes exactly one class of *false* mismatch: sorting fixes out-of-order retirements (Rule 1), program-order keying bridges the reference’s null uuids (Rule 2), mask-union aggregation reassembles split loads (Rule 3), the LSU probe with region filter restores sound load comparison (Rule 4), and volatile-CSR exclusion removes timing-only divergences (Rule 5). After alignment, any remaining difference outside the three declared exception classes is a real defect.

- **lockstep injection** (+LOCKSTEP_INJECT): one bit of one retirement is flipped; the lockstep comparator must catch it at the exact uuid/PC/lane;
- **DCR RAL injection** (+DCR_RAL_INJECT): a register-abstraction-layer mirror corruption must be caught by the register checker.

All guards must stay red on injection after any checker change—a discipline directly in the tradition of mutation-based checker qualification [17], applied end-to-end here to a SIMT lockstep flow. During the work, a nearly-accepted *vacuous* gate pass (negative tests reporting PASS when their injection plusargs were absent) was caught and codified into the rule: always verify the injection message, never the verdict. This discipline, rather than any single checker, is what allows the coverage and pass-rate numbers below to be taken at face value.

V. PER-INSTRUCTION LOCKSTEP FOR SIMT

A. Architecture

The DUT-side probe stream and the SimX per-retirement stream meet in a lockstep scoreboard that aligns both streams per (core, warp) and compares each retirement per active SIMT lane: PC, destination register, and written value, with a four-way outcome taxonomy (matched / field mismatch / data mismatch / orphan) and drain-empty checks on both sides so that dropped or extra retirements are caught (Fig. 2).

B. Five SIMT-specific alignment rules

A scalar-CPU RVVI flow never meets the following; each rule was forced by simulation evidence and is, we believe, transferable to any SIMT lockstep implementation.

1) *Retire order is not program order*: Within one warp, the commit arbiter retires whichever execution unit is ready first (VX_commit.sv:56–71); an earlier-issued instruction was observed retiring after two later ones. The functional model retires in strict program order. *Rule: align by the per-warp issue counter (uuid), sorted—never by position or cycle.*

2) *The golden model’s uuid is not a cross-key*: SimX leaves its retirement uuid at zero (Section X), so DUT and reference uuids cannot be matched directly. *Rule: the alignment key is per-(core, warp) program order; the DUT uuid’s high bits—which encode (CORE_ID $\ll$ NW_BITS) + wid (VX_uuid_gen.sv:40)—supply the (core, warp) attribution with no RTL change.*

3) *One instruction is not one retirement record*: A single load can appear as several commit records sharing one uuid with partial, overlapping thread masks (observed: masks 0xd then 0xe), because the LSU commits lanes as their memory responses arrive—distinct from clean SIMD-beat splitting. *Rule: aggregate DUT records by uuid with thread-mask union before comparing; never aggregate by start/end-of-packet flags.*

4) *Load data needs its own tap and its own soundness filter*: The commit arbiter’s data field is stale for loads (the LSU writes the register file on its asynchronous response path). A dedicated LSU writeback probe recovers true per-lane values—but a naive compare is *unsound*: loads of uninitialized stack or local memory legitimately differ between models (429 false mismatches on a known-good kernel). *Rule: compare a load lane only when the reference-model effective address lies in the verifiable data region and the golden value is not the initialization poison; defer everything else to the end-state check.* With this filter the load compare is exact and on by default (74 in-region lanes compared, 113 filtered, 0 false mismatches on the reference kernel).

5) *Performance-counter CSRs are model-divergent by definition*: mcycle/minstret/mhpmcounter* differ between a timing-accurate DUT and a functional model (observed off-by-one). *Rule: the golden model flags the machine-performance-monitor CSR range as volatile and the comparator excludes their data (PC and destination still checked)—the standard RVVI exclusion class.*

Beyond the five rules, the comparator declares three bounded, logged exception classes: the load-value deferral of Rule 4, the volatile-CSR exclusion of Rule 5, and an *fsqrt-only tolerance*. The DUT’s fsqrt.s is one ULP off correctly-rounded IEEE 754 (Section IX), so an fsqrt writeback is tolerated iff the two values are same-sign, finite, and within one ULP (fp32_within_ulp, keyed on an is_fsqrt flag exported by the reference); every tolerated comparison is tallied (n_fp_ulp_tol)—logged, never silent. All other floating-point operations compare bit-exact, and an fsqrt beyond one ULP, or a NaN/Inf mismatch, still fails. A two-pass reconvergence feed (armed with the load feed on +LOCKSTEP_LOADFEED) then forces the accepted DUT square-root result into the reference’s FP register file (re-NaN-boxed for the FLEN=32 build) and re-checks every

TABLE II

LOCKSTEP CONFIGURATION MATRIX (ALL LANE-EXACT: 0 MISMATCHES, 0 ORPHANS). TALLIES ARE RECORDED IN THE COMMITTED VERIFICATION DOCUMENTS; PER-RUN RAW LOGS WERE NOT RETAINED. THE 8W/4T POINT PREDATES THE RESET-RELAY AND PARAMETER-AUDIT FIXES AND DOCUMENTS THE COMPARATOR MECHANICS RATHER THAN CURRENT-DESIGN STATUS.

Configuration	Matched writebacks
1CL/1C/4W/4T (vector-add kernel)	1,035 / 1,035
1CL/1C/4W/4T (nested divergence)	2,668 / 2,668
1CL/2C/4W/4T	1,801 / 1,801
2CL/2C/4W/4T	3,333 / 3,333
1CL/1C/2W/2T	855 / 855
1CL/1C/8W/4T	1,423 / 1,423

downstream operation bit-exactly, so the accepted deviation cannot cascade into false errors; `fpu_test` and `fpu_mt` verify to residual zero under this handling (1,675 matched retirements; four `sqrt` reconvergences in the multi-threaded case). Each rule also carries a DUT precondition an adopter must re-derive: Rule 1 a unique per-warp issue counter that does not wrap within a run; Rule 2 a stable (core, warp) attribution key; Rule 3 records that may split per lane but share an identifier; Rule 4 a tap where true load values are observable; Rule 5 knowledge of the model-divergent CSR ranges.

C. Cross-configuration validation

The lockstep was validated lane-exact—zero mismatches, zero orphans—across a configuration matrix spanning $NCL \in \{1, 2\}$, $NC \in \{1, 2\}$, $NW \in \{2, 4, 8\}$, $NT \in \{2, 4\}$ (Table II), including a nested-divergence kernel (asymmetric $3v1 \rightarrow 2v1 \rightarrow 1v1$ splits) that exercised the mask-union aggregation through divergence and reconvergence with no comparator changes.

VI. VERIFYING RACY PROGRAMS: THE TWO-PASS LOAD-VALUE FEED

Fenceless multi-core programs have no architecturally defined result on a weakly coherent machine, so a per-instruction comparison against a functional model diverges legitimately: the two models resolve the same race differently. Rather than waive such programs, we make them verifiable (Fig. 3). A first pass records the DUT’s per-lane loaded values at the racy load sites, keyed by (core, warp, PC, occurrence); a second pass re-runs the reference following those values, so the models share the race outcome and any *remaining* difference is a real defect. This is a load-value feed in the RVVI sense, and it is not suppression: a residual not explained by a fed load stays a hard error.

On the flagship case—20 racy loads cascading into 138 divergent retirements at first pass—the residual collapses to **zero over 5,432 retirements** (5,314 matched, 118 data mismatches at pass one; 5,432/5,432 matched at pass two), upgrading the verdict from “UNVERIFIABLE” to *verified modulo the fed race resolutions*. The feed’s soundness rests on an assumption we

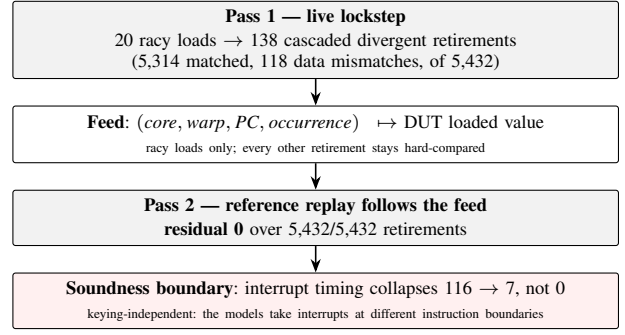


Fig. 3. The two-pass load-value feed on the flagship fenceless two-core case. Pass 1 runs the live lockstep and records the DUT’s resolution of every racy load, keyed by (core, warp, PC, occurrence) so that replays with different instruction interleavings still address the same load site. Pass 2 re-executes the reference model while feeding it those values, so both models share each race outcome and the residual becomes the verdict: zero mismatches over all 5,432 retirements. Feeding is not suppression—a difference not explained by a fed load remains a hard error. Asynchronous interrupts sit outside the fixed point: the residual there is proven keying-independent and is reported as classified, not forced green.

state rather than discharge in general: that a divergent in-region load is a genuine race resolution, not a DUT load-data defect. For the flagship case the assumption was confirmed by trace analysis—the first-divergence load reads a word still holding its pristine ELF initialization value (nothing had overwritten it before the DUT’s read) in a single-hart program running on four shared-memory cores, a genuine cross-core ordering race. The boundary is precise and we state it rather than hide it: for an *interrupt* test the same feed collapses 116 divergences to 7 but not to 0, because the two models take the interrupt at different instruction boundaries, so a same-PC occurrence count itself diverges. Re-keying the feed proved this is not an alignment artifact—the residual is identical. Interrupt-timing divergence therefore remains instruction-granularity unverifiable, while the end-state comparison still passes.

VII. STIMULUS

Directed kernels (~30, all compiled with the Vortex toolchain, all byte-exact against the reference): arithmetic and memory baselines; all 13 FPU operation classes; tensor-unit WMMA including multi-warp collective launches; nested-divergence towers; barriers under partial thread masks; `wspawn/tmc` sweeps; cache miss-status-holding-register (MSHR) and memory-pressure stress; a 232 KB-text kernel; a 256 KB high-entropy store stress; division corner cases; CSR writes under SIMT masks; and VOTE/SHFL operations.

Constrained-random: a riscv-dv [7] pipeline targeting rv32im, with GPU-specific post-processing (machine-mode CSR stripping, `ecall`→`ebreak`) and end-state verification; Twelve riscv-dv profiles were brought up; two are excluded as *unimplementable* on this DUT—unaligned-load/store (no misaligned support, Section IX) and illegal-instruction (no trap architecture)—and one was later dropped as a byte-identical duplicate of another profile, leaving nine generator profiles

(twelve seeded instances run in the suite). Jump-stream generation is sanitized to keep the generator’s deliberate JALR-LSB probing away from an RTL deviation it would otherwise trip (finding R1); the deviation itself is reported, not hidden.

SIMT-aware generation (simtgen): an original random C-kernel generator targeting the microarchitectural axes that scalar generators cannot reach. Its knobs are derived from the DUT’s own configuration space (warp/thread counts, LSU coalescer window, local-memory bank count, IPDOM stack depth), informed by the generation-axis taxonomy published by FuzzGPU [3] but implemented against our own checking stack; no FuzzGPU code is used (an evaluation of reuse, rejected for engineering reasons, is documented in the artifact). `simtgen` implements four axes: divergence (structured branch trees with controlled split depths and asymmetric masks), memory (stride/scatter/bank-hostile access patterns), barrier topology (single-warp barriers, deadlock-safe by construction; `wspawn` is deliberately never emitted—it is a runtime-only bootstrap primitive with its own structural waiver), and VOTE/SHFL (all eight intrinsics per program). The barrier and VOTE/SHFL axes closed no new coverage—their targets (`cp_vote_shfl_op` 8/8, `cross_sfu_threads`) were already fully covered by dedicated directed kernels (`vote_shfl`, `bar_masks`, `sfu_masks`); their contribution is generator completeness and seed-diversity robustness evidence, the same category of result as the seed farm below. `simtgen`’s measured effect is quantified in Section VIII; a ten-seed sweep across all nine riscv-dv profiles (90 additional distinct programs, content-hash verified, all passing) produced *no measurable functional-coverage gain* (every category bit-identical except toggle at +0.06%)—a robustness result we report as such, not as coverage progress: coverage saturates inside the stimulus’s reachable region, and the remaining work is stimulus of a different *kind*, not more samples of the same kind.

Protocol stress: the AXI throttle, flood, and error-injection modes of Section III, plus host/DCR launch-space sweeps.

VIII. COVERAGE METHODOLOGY AND RESULTS

A. A three-layer coverage model

The functional model is organized in three layers with different owners and deliberately different blind spots, and their databases are never merged:

- 1) **L1 — ISA coverage** (third-party riscvISACOV layer): 80 active covergroups over RV32I/F/M/Zicsr/Zifencei. Sampling *every active lane as a hart* (4,581 samples) was proven to cover exactly the same bin set as lane-0-only sampling (1,677 samples) on our kernels—an empirical demonstration that ISA coverage is blind to the SIMT axis, which is why L2 exists. A closed gap-hunt (frozen 2026-09-06) leaves 78/80 covergroups real and 429/516 target bins hit (83.14%; 89.28% weighted) after register-index exclusions; two covergroups are permanently empty by construction.
- 2) **L2 — SIMT microarchitecture** (ours; the layer this paper contributes): probe-sampled covergroups for

warp divergence crossed with IPDOM stack depth (divergence_cg), reconvergence, barrier scope and events, `tmc/wspawn` behavior, scratchpad bank conflicts (`lmem_bank_cg`), memory-coalescing classes (`coalesce_cg`), cache events, register hazards, and commit-beat shapes (`beat_cg`), alongside instruction-class groups per execution unit.

- 3) **L3 — system and protocol** (ours): SVA assertion and directive coverage, AXI transaction space, DCR/host launch spaces, and system-state crosses.

B. Closure discipline

Three rules govern closure. (1) *Exclusions are structural, RTL-cited, and machine-generated:* a per-configuration generator emits every waiver with a `file:line` citation (e.g., a write-response stability assertion is unreachable because the adapter hardwires `m_axi_bready = 1'b1`, `VX_axi_adapter.sv:313`), and a blocking *hits-invariant* merge gate verifies that every waived bin had zero hits in the unwaived bank—on its first execution this gate caught two real waiver defects, one of which had been active for weeks. Exclusions are keyed to the configuration—e.g., the global-barrier path is excluded only at single-core, where it is structurally unreachable, and kept at ≥ 2 cores. (2) *Reachable-but-unhit is left red:* bins that stimulus could reach but did not are reported uncovered, never waived. (3) *Ceilings are root-caused:* the toggle plateau was traced, per sub-module, to a read-only icache data port (51,340 bins, 55.7% sub-module toggle; 22,730 bins missing—26.4% of the *entire* toggle gap—traceable to `VX_socket.sv:106` tying `.WRITE_ENABLE(0)`) and constant high address bits; an adversarial maximum-entropy kernel moved aggregate toggle by +0.02%, establishing the ceiling as structural. Coverage banks are reported *per configuration*: merging coverage databases across topologies was shown to be invalid (instance-set inflation deflates by-instance percentages).

C. Results

Table III summarizes the banked configurations. All totals are stated with their provenance and labels:

D. SIMT-layer closures and the generator’s measured effect

The SIMT layer is where the generator paid. Before `simtgen`, three coverage targets were *believed* open: the depth-4 IPDOM divergence bin (`divergence_cg.cp_split_depth`, 3/4—depth 3 never hit by any of 50 seeds), the bank-conflict bin of `lmem_bank_cg.cp_bank_conflict` (0/3, never sampled in 70,498+ cycles), and the coalescing classes of `coalesce_cg.cp_coalesce_kind`. Deliberate `simtgen` programs closed the first two (Fig. 4): structured divergence towers hit depth 4 (4/4), and bank-hostile access patterns (all lanes of a warp addressing one bank by construction) surfaced real conflicts; both closures are folded into a suite bank (2026-09-09). Three honesty notes are attached to this result and travel with it wherever it is cited:

TABLE III

COVERAGE BANKS (QUESTASIM; PER-CONFIGURATION, NEVER BLENDED). ALL ROWS ARE THE 2026-08-16 *pre-relay-fix* BANKS; THE RESET-FIXED 1CL BANK (51 RUNS, 2026-08-18) TOTALS THE SAME 94.7% WITH BRANCH 94.53% AND TOGGLE 83.34%—INDICATIVE ONLY, TWO VARIABLES CHANGED (§VIII-E). A RELAY-FIXED 2CL BANK TAKEN AFTER THIS TABLE WAS FROZEN (110 RUNS, 0 FAILURES, 93.4% COVERAGEGROUP BINS / 94.3% TOTAL, ON THE GROWN POST-REMEDIATION COVERAGE MODEL) RESTORES CROSS-CONFIGURATION COMPARABILITY; THE PRE-FIX COLUMNS ARE RETAINED FOR PROVENANCE.

Metric	1CL/1C/4W/4T	2CL/2C/4W/4T
Covergroup bins (raw)	370/377 = 98.1%	989/1032 = 95.8%
Covergroup (weighted)	99.8%	99.5%
Statement	98.1%	98.3%
Branch	95.1%	95.7%
Condition	90.4%	88.8%
Toggle	82.8%	80.5%
Assertion	96.9%	98.9%
Directive	100.0%	100.0%
Total	94.7%	94.6%
Runs passing	50/50 [‡]	50/50
Coverage instances	2 256	8 275

[‡] *Simulation runs*, not programs: 49 distinct programs, one run twice under two bus modes.

A third bank with shared L2/L3 enabled totals 93.2% (51 runs, all passing), also on the pre-fix design. A relay-fixed L2/L3 bank (seven-kernel cache-focused suite, 2026-09-11) replaces the prior passthrough waivers with genuinely exercised shared-hierarchy coverage: 85.1% total on the correspondingly enlarged denominator, with live cache-probe hit counts verified recursively.

- the bank-conflict bin’s initial “gap” was in fact a *coverage-model defect* (OBS-060): the coverpoint had been defined over *accepted* requests, but the local-memory crossbar admits exactly one winner per bank per cycle—“two accepted requests on one bank in one cycle” was structurally impossible, and deliberate bank-hostile stimulus proved it by producing zero conflicts. Reclassifying the coverpoint onto *offered* requests made the bin reachable and simultaneously honest (169 conflict hits, 61 no-conflict, 70,460 idle); the closure credit is therefore shared between the probe fix and the generator. A coverage definition must itself be validated against the RTL’s structure; the stimulus gap and the taxonomy defect were disentangled by experiment.
- the coalescing classes, which an earlier isolated-merge report attributed to simtgen, were re-examined and found already covered by the existing coalescer probe and kernels before simtgen existed—an attribution correction we report rather than quietly keep.
- the barrier and VOTE/SHFL generator axes closed nothing because their targets were already fully covered by directed kernels (Section VII); their value is generator completeness and seed-diversity robustness. The marginal picture per generator kind: ninety additional riscv-dv programs closed zero bins (the seed-farm result), while 57 simtgen programs (divergence and memory axes) closed the divergence and bank-conflict targets—stimulus *kind*, not volume, moves this coverage model.

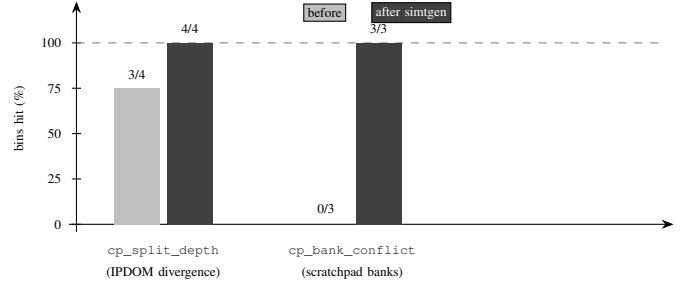


Fig. 4. Effect of axis-directed generation on the two SIMT targets it genuinely closed (bins hit, as a percentage of the coverpoint’s bin count). Before simtgen (light bars): depth-4 IPDOM divergence was unreachable by any of 50 random-program seeds (3/4 bins); bank conflicts never occurred in 70,498+ sampled cycles (0/3—its root-caused to a coverpoint-definition defect, since the crossbar admits one winner per bank per cycle). Deliberate divergence-tower and bank-hostile kernels (dark bars) closed both to 100%, now folded into a suite bank (2026-09-09). A third target previously reported here (coalescing classes) was re-examined and found already covered by the existing coalescer probe and kernels before simtgen existed—an attribution correction to our earlier isolated-merge reporting.

E. Provenance of the design under verification

The design is an open-source register-transfer model pinned at a specific revision, *with eighteen locally modified files in the synthesizable register-transfer tree* (support and co-simulation glue outside that tree is additionally modified, and the environment itself is new code). Most changes are simulator-compatibility fixes made during bring-up. We disclose this because a verification result is a statement about a specific artifact, and “upstream, unmodified” would not describe what we ran.

One of those modifications turned out to matter, and its history is instructive. A library counter shipped with two assertions checking for overflow and underflow. They fired during bring-up and were guarded off—rewritten to skip whenever their inputs were unknown—so that the environment could run at all. That guard remained for months. On restoring the original assertion and root-causing why it fired, we found a genuine defect (R10, Section IX-A). We replaced the relay with an asynchronous-assert, synchronous-deassert synchronizer; the counter assertions then pass with no guard at all—twelve firings become zero—and the local modification to that counter was retired in favor of the unmodified upstream file.

Two consequences deserve emphasis. First, **the assertion had been correct**: what was silenced was a real property of the design, not a testbench nuisance, and the cost was months of an unobserved unknown-value window. Second, **fixing the defect reduced a coverage number**: branch coverage fell from 95.1% to 94.5% because modules behind a relay had been executing their normal-operation paths during the unknown-reset cycle, and those executions were *counted as covered branches*. We report the lower figure as the correct one. (Two variables changed between those measurements, so the delta is indicative rather than isolated.)

TABLE IV
RTL FINDINGS (SUMMARY). THE R1 SPEC DEVIATION WAS INDEPENDENTLY FOUND BY FUZZGPU [3] IN THE ISS COMPONENT (S2, PR #339); THEIR RTL FINDINGS (X1–X3) ARE DISJOINT FROM OURS; R2 WAS INDEPENDENTLY FIXED UPSTREAM.

ID	Finding	Class	Disposition
R1	JALR target LSB not cleared; odd PC propagates via AUIPC	Bug (ISA)	needs RTL fix; stimulus sanitized
R2	STALL_TIMEOUT uses $1 \times N \equiv 1$; watchdog never scales	Bug (latent)	fixed; <i>independently fixed upstream</i>
R3	Misaligned access: no trap; silently retargeted/torn	Hazard	expected per SW contract; gated
R4	Core self-starts from reset; DCRs have no reset value	Hazard	worked around (reset handshake)
R5	Per-warp out-of-order commit	Quirk	handled (§V-B)
R6	One load $\rightarrow$ multiple commit records, overlapping masks	Quirk	handled (§V-B)
R7	uuid encodes flat core id + warp id	Quirk	exploited (§V-B)
R8	Load data not observable at commit arbiter	Observability	closed (LSU probe)
R9	Write path fire-and-forget; <i>no AXI error path</i>	Characteristic	cited; fault-injected (166)
R10	Reset relay: reset registered in an unreset flop; one-cycle X window	Bug (X source)	fixed; still upstream

IX. FINDINGS I: RTL OBSERVATIONS

Every finding is maintained in an evidence-cited register shipped with the work (one entry per finding; what was seen, `file:line` and/or run evidence, classification, disposition). Table IV summarizes the RTL findings as R1–R10; two are developed below, and further verified defects are noted afterwards.

A. An unknown reset for one cycle, found by restoring a silenced assertion (R10)

The design distributes reset through a relay module. That module registers the incoming reset in a flip-flop which has no initial value and which nothing resets:

```
'PRESERVE_NET reg [R-1:0] reset_r; // no initial value
always @(posedge clk) begin
    reset_r[i] <= reset; // nothing resets THIS flop
end
assign reset_o[i] = reset_r[i / F];
```

Its output is therefore unknown from time zero until the first clock edge (Fig. 5), so every module instantiated behind a relay observes an *unknown reset for one cycle*. A reset-conditional in such a module takes its non-reset branch—`if (X)` is not true—so logic intended to be held in reset executes, and any assertion inside that

branch evaluates on unknown operands.

The path to it is the point. A library counter in the same design ships with assertions checking for overflow and underflow. Those assertions fired during bring-up, at 5, 15 and 25 ns, on the instruction- and data-cache miss-status counters. They were guarded off—rewritten to skip whenever their inputs were unknown—so that the environment could run, and

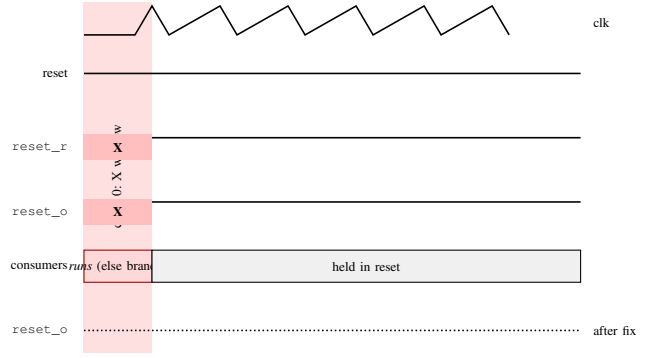


Fig. 5. The reset-relay defect (R10) as a timing picture. The relay registers `reset` in a flip-flop that has no initial value and that nothing resets, so until the first clock edge (shaded cycle 0) its output—and therefore the reset of every module behind the relay—is unknown (X). Because `if (X)` takes the else branch, reset-conditioned consumer logic executes its *normal-operation* paths during that window, and assertions evaluate on unknown operands: this is exactly what the (guarded-off) counter assertions had been reporting. With the *async-assert/sync-deassert* fix (bottom trace), `reset_o` asserts with reset from time zero and the twelve counter-assertion firings become zero—using the unmodified upstream file.

the guard stayed for months. Restoring the original assertion and asking why it fired produced the diagnosis above. **The assertion had been correct the entire time:** it was reporting a real property of the design, and what had been suppressed was the report, not the problem.

We replaced the relay with an asynchronous-assert, synchronous-deassert synchronizer, so that the relay output asserts whenever reset asserts regardless of flip-flop state. The counter assertions then pass with no guard at all—twelve firings become zero—and the local modification to the counter was retired in favor of the unmodified upstream file. A fifty-one-run regression passes with the assertions armed, and the fault-injection guards were re-confirmed to fail on injection. The defect is present in the current upstream release.

Two observations generalize beyond this design. First, **a checker disabled to make a bench run is a checker whose findings are lost**, and the loss is silent and open-ended: here it was months, and the affected surface was every module behind a relay, not merely the one that happened to carry an assertion. Second, **fixing the defect reduced a coverage number** (Section VIII-E): no coverage metric can report that some of its own coverage came from a state that cannot legitimately arise.

B. JALR does not clear the target LSB (R1)

The RISC-V specification [12] requires JALR to clear the least-significant bit of the computed target. Vortex omits the clear (`VX_alu_int.sv:222`: the branch destination is the raw `rs1+imm`), and—having no trap architecture—cannot raise the instruction-address-misaligned exception the specification prescribes for a misaligned target on a non-compressed core. In the debug build (`PC_BITS = XLEN`, identity PC conversion) the odd bit survives as the *architectural* PC. Fetch silently word-aligns (`VX_fetch.sv:101`), so execution continues on correct instruction words—but the

skewed PC *does* reach architectural results: every `auipc/la` computes $rd = PC + imm$ and inherits the skew, link-register writes accumulate it across chained jumps (observed $+1 \rightarrow +3$), and downstream loads/stores go misaligned, cascading into R3. The trigger is spec-legal stimulus: `riscv-dv` deliberately sets the JALR base LSB expecting the architectural clear, so roughly half of generated jumps derail—in a 12-profile suite, every profile fired misaligned-access assertions (30–7,616 per run). In the release build (`PC_BITS = XLEN$-2`) a $+1$ target is masked away by representation—spec-correct *by accident*—while a $+2$ target would silently word-align where the specification demands a trap. The fix is a one-line `\&\, \, \^{\{ } 1` at the destination adder; because the reference model deliberately mirrors the no-clear behavior, the fix must un-mirror both models together. FuzzGPU [3] found the same deviation independently from a fuzzing flow entirely different from ours (S2, upstream PR #339, CWE-682—filed against the project’s instruction-set simulator; their RTL findings X1–X3, at a different pin, are disjoint from ours)—independent corroboration of the *deviation*, though not of its RTL embodiment.

C. Further verified defects

fsqrt.s is one ULP off correctly-rounded IEEE 754 [13] (OBS-014): a lockstep lane-0 data mismatch (DUT `0x3fef7750` vs. the SoftFloat-derived reference `0x3fef7751`) localized by instruction and lane to the square-root unit; the adjacent `fdv.s` compared exact. The end-state scoreboard’s FP tolerance absorbs one ULP by design—*only* the per-instruction comparison surfaces it. **A second watchdog defect** (OBS-017): beside R2’s macro bug, the memory scheduler’s timeout `localparam` is denominated in simulation-time picoseconds, yielding a $\sim 1,000$ -cycle budget on our timescale and flooding L2/L3-enabled runs with 22,187 false-alarm messages while functionally clean (2,789/2,789 lockstep match). **No AXI error path** (OBS-057/R9): the AXI adapter hard-asserts that responses are OKAY; injecting `SLVERR/DECERR` on every seventh transaction produced exactly 166/166 assertion firings and no recovery path—an evidence-based upgrade of a waiver into a measured design limitation, consistent with the missing trap architecture.

X. FINDINGS II: REFERENCE-MODEL OBSERVATIONS

A central lesson of this work: *a lockstep flow verifies the reference model as much as the DUT*. SimX is maintained by the same project as the RTL, so shared assumptions can hide shared bugs; the per-instruction compare surfaced defects in the model itself.

A. Misaligned instruction fetch, found by the lockstep itself

The reference read instructions at the exact byte program counter. A computed indirect jump to an odd target—which the RTL tolerates, because fetch word-aligns—made the reference read across an instruction boundary and abort, classifying the run unverifiable. The lockstep’s own divergence report localized it. Word-aligning the fetch while leaving the architectural

PC odd (matching the DUT) recovered those runs. The *obvious* alternative—masking the JALR target in the reference—was tried and *reverted with evidence*: it produced 21,968 phantom PC mismatches, because it fixed the model to the specification rather than to the DUT. This is the clearest case for treating the golden model as a verification target in its own right: the defect was in the reference, and only a per-instruction comparison could have attributed it there.

B. Abort surface and SIMT-divergent CSR semantics

SimX’s unknown-input behavior is to abort—56 `std::abort` sites by disassembly (35 decode, 21 execute), the majority reachable through the disassembly pretty-printer rather than the decoder core: a run can be unverifiable because the golden model could not *name* an instruction. The lockstep upgrades such verdicts to “byte-exact for all 17,664 retirements before the abort.” Separately, a SIMT-divergent CSR access produced 109 genuine data mismatches: the reference serialized lane effects over the shared per-warp `fcsr` while the RTL reads-broadcast and writes lane 0. The reference was corrected to read-once/write-lowest-active-lane semantics; the RTL quirk is deliberately *not* mirrored, and is documented as a visible architectural choice.

XI. LIMITATIONS AND THE SOUNDNESS BOUNDARY

A. Asynchronous interrupt timing

The interrupt-random test at two clusters collapses from 116 cascaded divergences to 7 under the load feed—and does not reach zero. The residual is proven *keying-independent*: re-keying the feed from a per-warp ordinal to $(cid, wid, PC, occurrence)$ leaves it identical, disproving feed-alignment artifacts. The cause is genuine: the timing-accurate DUT and the functional model take asynchronous interrupts at different instruction boundaries, so an interrupt-affected path executes a different number of times; no load-data feeding can align *when* an interrupt fires. The end-state compare against the post-feed model passes, so the disposition is: end-state VERIFIED, instruction-granularity residual classified—not forced green. This defines the method’s soundness boundary: *two-pass trace replay is a fixed point for data-only divergence; asynchronous-input timing requires a step-follower reference*.

B. Control-flow-steering races

When racy loaded bytes steer control flow (a random jump test), pass-2 replay walks a different path and meets *fresh* racy loads the pass-1 trace never keyed (residual 15, thirteen of them load-class; the post-feed end-state compare is clean). Two-pass replay has no fixed point when races feed branches; an iterated (bounded fixed-point) feed or a step-follower is required. The verdict is left honestly red.

C. Structural blindness of differential testing

Differential testing is structurally blind to a fault in the *stimulus*: DUT and reference execute the same binary, so a mis-compiled kernel is mis-executed identically and every checker passes while the kernel verifies nothing—observed

concretely when a tensor kernel’s compile-time geometry silently mismatched the RTL configuration (OBS-028/OBS-029). Equivalence validates only what *differs* between the models; everything shared (binary, memory image, launch geometry) requires independent assertions. We closed exactly one such input (kernel geometry asserted at elaboration, injection-proven) and report the class as open.

D. Other limitations

- **Reference-model independence.** SimX is not independent of the DUT. The Spike audit covers the scalar subset only (warp 0 / lane 0); *the SIMT axis has no independent reference, and this is a structural ceiling rather than an unfinished task.*
- **Simulator dependence.** All results are on one commercial simulator (QuestaSim). Open-source simulation of UVM has matured rapidly—recent Verilator elaborates upstream UVM 2017 with constrained randomization—but SystemVerilog functional coverage (covergroups) remains unsupported there, so the coverage-driven half of this methodology cannot yet run on an open-source simulator; the lockstep and end-state checking halves are, in principle, portable today.
- **D-extension elaboration, scoped by inspection.** The D extension is elaborated at the primary configuration and MISA reports D=1 (FLEN=64, confirmed by an elaboration probe) although the toolchain builds RV32IMAF and the reference model models F only—unintentional-by-documentation as an RTL fact. Its coverage impact is bounded by code inspection and confirmed by an EXT_D-off recompile with a coverpoint-by-coverpoint bin diff: zero scored bins change, the entire delta confined to two weight-0 coverpoints already excluded from every reported percentage. The coverage model’s only D-specific bin (float-double conversion) is config-aware-excluded at RV32 (ignore_bins keyed on XLEN), the ISA layer compiles no D plan, and no stimulus can emit a D operation—so no functional bin is diluted. The elaborated-but-unexecuted D logic sits only in code-coverage denominators, which makes the reported totals *conservative* (they understate an F-only build), never inflated.
- **Per-mnemonic coverage soundness.** The frozen banks predate the discovery that several instruction-class encodings alias (4'b1011 decodes as both a zero-conditional op and EBREAK); per-mnemonic ALU identity in the frozen bank is therefore unsound, and per-mnemonic claims are drawn only from post-remediation measurements.
- **Structural coverage ceilings.** Toggle coverage plateaus for root-caused structural reasons; we report the true value rather than gaming it.
- **No trap architecture.** Exception-path verification is unimplementable on this DUT; the corresponding generator profiles are excluded as unimplementable rather than skipped silently.

- **Verified build.** Findings are stated against the debug build at one RTL pin; the release build changes the visibility (not the presence) of finding R1. The configuration-matrix tallies of Table II are documented results; their raw per-run logs were not retained.

XII. TOWARD TAPE-OUT

The work reported here is *front-end functional verification* of an RTL model. Silicon sign-off requires substantially more, and the two categories below are deliberately separated because they demand different tooling, different skills, and—critically—different claims. Neither list is a statement of partial progress: every item is currently absent unless noted.

1) *Remaining front-end (RTL functional) work:* (1) Stimulus diversity, not volume—the seed-farm negative result shows coverage saturating inside the scalar generator’s reach; the remaining work is stimulus of a different *kind* (privileged behavior, exception stimulus, bus errors—partly delivered by the error-injection mode—and the two unimplemented simt-gen axes). (2) Error and exception verification beyond the injection gate. (3) Formal property verification on arbitration-heavy control (cache miss-status handling, commit arbitration), where dynamic simulation is weakest and state space is small enough for proof; the IPDOM reconvergence stack (push/pop balance, no underflow, reconvergence restores entry mask) is the natural first target. (4) X-propagation and reset randomization—R10 proves the bug class exists in this design, and several unreset elements have been identified structurally but never exercised under randomized reset. (5) Configuration-matrix breadth beyond three sampled points. (6) Coverage-model provenance: the L2/L3 layers are self-authored rather than traced to a specification document, so closure measures the model, not the specification. (7) An independent SIMT reference—structurally absent, as above.

2) *Back-end and ASIC sign-off (entirely out of scope):* Gate-level simulation with back-annotated timing; static timing analysis across PVT corners; DFT (scan, ATPG, MBIST); CDC/RDC analysis with metastability modeling; lint and structural sign-off; low-power verification; physical-design closure; equivalence checking between RTL, netlist, and post-layout; and a post-silicon path back into this regression. A useful way to read the two lists: the front-end items would raise the strength of the claims made in this paper, whereas the back-end items are prerequisites for a different claim entirely—that the design is manufacturable and will function in silicon. Nothing in this work speaks to the latter.

XIII. CONCLUSION

We presented a UVM-based methodology that brings reference-model verification—the standard of care for RISC-V CPU cores—to a SIMT GPGPU, together with the first published SIMT functional-coverage model for RTL GPU verification. The environment is complete and configurable: role-inverted agents around a bus-master DUT, a per-configuration co-built functional reference over DPI-C, a passive RVVI-style observability layer, bidirectional end-state checking, and an

injection-proven non-vacuity discipline behind every verdict. The lockstep layer contributes five SIMT alignment rules (each with a stated DUT precondition) and a two-pass load-value feed that turned an architecturally-undefined racy multi-core program from unverifiable into verified modulo the fed race resolutions (residual 0 over 5,432 retirements), while honestly delimiting its own soundness boundary at asynchronous interrupt timing. The closure methodology—machine-generated, gate-checked exclusions, root-caused ceilings, residual bins dispositioned rather than waived—carried the primary configuration to 98.1% coverage bins / 94.7% total, and the SIMT coverage layer was closed by an axis-directed generator (the divergence and bank-conflict closures are folded into a suite bank; the coalescing attribution and the coverage-model defect found on the way are reported with their evidence).

The checking depth paid for itself in findings on both sides of the comparison: in the RTL, a JALR ISA deviation with architectural consequences (independently corroborated by concurrent fuzzing work), a non-scaling watchdog macro (independently fixed upstream), a silent misaligned-access corruption mode, a missing AXI error path, and a reset-relay X window found by restoring a silenced assertion; in the reference model, a misaligned-fetch bug that had been silently discarding verification results, found by the lockstep itself. Both registers ship with the work, each entry evidence-cited and dispositioned. We believe the methodology—and its central lesson, that the reference model must be verified alongside the DUT and the coverage model alongside both—transfers directly to other open-source SIMT designs, and fills the half of the open-GPU verification problem that fuzzing does not address.

ACKNOWLEDGMENT

The authors thank the Vortex research group at the Georgia Institute of Technology for developing and openly maintaining the GPGPU platform on which this work is built, and the maintainers of the open verification assets (core-v-verif, riscv-dv, RVVI, riscvISACOV, Spike) leveraged by the environment.

REFERENCES

- [1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics,” in *Proc. 54th IEEE/ACM Int. Symp. Microarchitecture (MICRO-54)*, 2021, pp. 754–766.
- [2] F. Elsabbagh, B. Tine, P. Roshan, E. Lyons, E. Kim, D. E. Shim, L. Zhu, S. K. Lim, and H. Kim, “Vortex: OpenCL compatible RISC-V GPGPU,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.
- [3] Z. Gao, J. Wang, Q. Zhou, L. Shan, J. Hu, X. Jia, and Z. Lv, “Fuzzing open-source GPU hardware with SIMT program generation,” in *Proc. 35th USENIX Security Symposium (USENIX Security ’26)*, Baltimore, MD, Aug. 2026, pp. 2465–2484.
- [4] Y. Xu *et al.*, “Towards developing high-performance RISC-V processors using agile methodology,” in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO-55)*, 2022.
- [5] OpenHW Group, “core-v-verif: functional verification project for the CORE-V family of RISC-V cores,” [Online]. Available: <https://github.com/openhwgroup/core-v-verif>
- [6] OpenHW Group / Imperas, “RVVI: RISC-V Verification Interface,” [Online]. Available: <https://github.com/riscv-verification/RVVI>
- [7] CHIPS Alliance, “riscv-dv: SV/UVM based instruction generator for RISC-V processor verification,” [Online]. Available: <https://github.com/chipsalliance/riscv-dv>
- [8] RISC-V International, “Spike, the RISC-V ISA simulator,” [Online]. Available: <https://github.com/riscv-software-src/riscv-isa-sim>
- [9] Imperas Software, “ImperasDV: processor verification IP with lock-step compare,” [Online]. Available: <https://imperas.com/imperasdv>
- [10] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020, 2020.
- [11] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2017.
- [12] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V Foundation, 2019.
- [13] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.
- [14] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson, “The design and implementation of a verification technique for GPU kernels,” *ACM Trans. Program. Lang. Syst.*, vol. 37, no. 3, pp. 10:1–10:49, 2015.
- [15] G. Li, P. Gopalakrishnan, I. Ghosh, S. P. Rajan, and G. Gopalakrishnan, “GKLEE: Concolic verification and test generation for GPUs,” in *Proc. 17th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2012, pp. 215–224.
- [16] C. Cummins, P. Petoumenos, H. Leather *et al.*, “Compiler fuzzing through deep learning,” in *Proc. 27th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2018, pp. 95–105.
- [17] L. Di Guglielmo, F. Fummi, and G. Pravaddelli, “Vacuity analysis for property qualification by mutation of checkers,” in *Proc. Design, Automation & Test in Europe (DATE)*, 2010, pp. 262–267.
- [18] T.-T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, M. B. Glick, and S. W. Keckler, “NVBitFI: Dynamic fault injection methodology for GPU applications,” in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2021, pp. 64–76.
- [19] Y. A. Manerkar, D. Lustig, M. Martonosi, and M. Pellauer, “RTLCheck: A property checking approach for custom memory consistency models,” in *Proc. 50th IEEE/ACM Int. Symp. Microarchitecture (MICRO-50)*, 2017, pp. 713–725.
- [20] D. Lustig, S. Sahasrabudhe, and M. Giroux, “A formal analysis of the NVIDIA PTX memory model,” in *Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 253–266.
- [21] T. L. Sharma, M. Bauer, and A. Aiken, “Verification of producer-consumer synchronization in GPU programs,” in *Proc. 36th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2015, pp. 88–98.
- [22] lowRISC, “OpenTitan: open source silicon root of trust,” [Online]. Available: <https://github.com/lowRISC/opentitan>
- [23] Siemens EDA, “QuestaSim,” [Online]. Available: <https://eda.sw.siemens.com/en-US/ic/questa/simulation/>
- [24] Verilator open-source project, [Online]. Available: <https://www.verilator.org>